Universidad Rey Juan Carlos

Escuela Técnica Superior de Ingeniería Informática

Práctica 1: Captura y Representación de decisiones de diseño

Diseño y Arquitectura del Software - Curso 2026/2027

Rafael Capilla, Micael Gallego

Grupo 3

Rim Afoud
Sergio Alejandro Romero López
Ignacio Roncero Medina
Lucca Manfredotti García
Neville Gil Ortiz

Correo del responsable n.gil.2024@alumnos.urjc.es

Tabla de contenido

1. Integrantes del equipo y roles 2
2. Introducción 4
3. Desarrollo de la práctica 4
    3.1 Requisitos funcionales 4
    3.2. Requisitos de dominio 6
    3.3 Toma de decisiones 6
Repositorio GitHub 8

1. Integrantes del equipo y roles

Arquitectos Software Senior (ASS):

- Rim Afoud
- Sergio Alejandro Romero López

Arquitectos Software Cognitivos (ASC):

- Lucca Manfredotti García
- Ignacio Roncero Medina

Arquitecto Software Junior (ASJ):

- Neville Gil Ortiz

2. Introducción

En este documento se recoge el análisis y las decisiones de diseño y arquitectura tomadas para el desarrollo de la práctica, en el contexto de la asignatura Diseño y Arquitectura del Software.

A lo largo del trabajo se identifican los requisitos funcionales y de dominio del sistema, que permiten comprender las necesidades y restricciones del problema planteado. A partir de estos requisitos, se analizan diferentes alternativas arquitectónicas y se justifican las decisiones adoptadas para definir una arquitectura que responda adecuadamente a las necesidades del sistema.

3. Desarrollo de la práctica

3.1 Requisitos funcionales

Identificador	Nombre	Descripción
RF1	Discriminación de clientes	El sistema deberá poder distinguir los tipos de dispositivos en los que está siendo usado.
RF2	Módulo de suscripciones	El sistema deberá contar con un módulo de suscripciones que maneje las suscripciones de los usuarios.
RF3	Alta anónima	El sistema debe permitir a un lector suscribirse a una o varias categorías de noticias de forma anónima.
RF4	Baja anónima	El sistema debe permitir a un lector darse de baja a una o varias categorías de noticias de forma anónima.
RF5	Categorías de noticas	El sistema permitirá clasificar las noticias en las siguientes categorías: Política, economía,

		sociedad, nacional e internacional.
RF6	Módulo de pagos	El sistema deberá tener un módulo de pagos online que gestione los ciclos de facturación
RF7	Modalidades de facturación	El sistema debe ofrecer tres modalidades de pago: Trimestral (70€), semestral (130€) o anual (190€).
RF8	Pago recurrente	El sistema deberá cobrar de forma recurrente al final de cada ciclo de la modalidad de facturación.
RF9	Reintento de pago	El sistema deberá en caso de que el pago falle, reintentarlo hasta un máximo de 3 veces.
RF10	Ejecución del cobro	El sistema ejecutará cada cobro de una suscripción mediante una pasarela de pago y registrará si ha sido aceptado o rechazado.
RF11	Módulo de estadísticas	El sistema deberá contar con un módulo que se encargue de recoger las siguientes estadísticas: Noticias más leídas, tiempo de permanencia, tasa de rebote y tasa de conversión.
RF11.1	Noticias leídas	El sistema deberá recoger estadísticas de cuantas lecturas recibe una noticia.
RF11.2	Tiempo de permanencia	El sistema deberá recoger estadísticas sobre el tiempo que un usuario tarda o dura en interactuar con elementos de la página.
RF11.3	Tasa de rebote	El sistema deberá recoger estadísticas sobre los usuarios que abandonan la página sin interactuar con ella.
RF11.4	Tasa de conversión	El sistema deberá recoger estadísticas sobre los usuarios que completan una acción

		necesaria como suscribirse o leer una noticia.

3.2. Requisitos de dominio

Identificador	Nombre	Descripción
RD1	Bases de datos	El sistema deberá incorporar bases de datos separadas, una para los administradores y redactores, y otra para los suscriptores.
RD2	Pasarela de pago STRIPE	El sistema deberá utilizar STRIPE como pasarela de pago online.
RD3	Dos bases de datos separadas	El sistema deberá disponer de dos bases de datos independientes, ambas implementadas en PostgreSQL.
RD4	Comunicación HTPP/REST	El sistema deberá realizar las peticiones entre los clientes y el servidor mediante protocolos HTTP/REST

3.3 Toma de decisiones

Selección Estilo Arquitectónico

Durante esta iteración se plantearon varios estilos arquitectónicos que darían lugar a la arquitectura final.

En un primer lugar, los Arquitectos Software Senior plantearon la posibilidad de emplear el estilo Por Capas, que separaría los componentes y módulos del sistema de forma ordenada. Los requisitos de nuestro problema se agrupan de forma natural en responsabilidades distintas:

- La interfaz web y la discriminación de clientes (RF1) quedarían en la capa de presentación.
- Los módulos de suscripciones (RF2, RF3, RF4), pagos (RF5, RF6) y estadísticas (RF8, RF8.1 a RF8.4) quedarían en la capa de lógica de negocio.

- Las dos bases de datos PostgreSQL separadas (RD1, RD3) quedarían en la capa de datos. Esto encaja además con que las suscripciones sean anónimas (RF2, RF3), ya que los datos de los suscriptores no se mezclan con los de administradores y redactores.
- La ejecución de los cobros mediante la pasarela de pago, Stripe (RF7, RD2), quedaría en una capa de servicios externos, que aísla esta dependencia del resto del sistema.

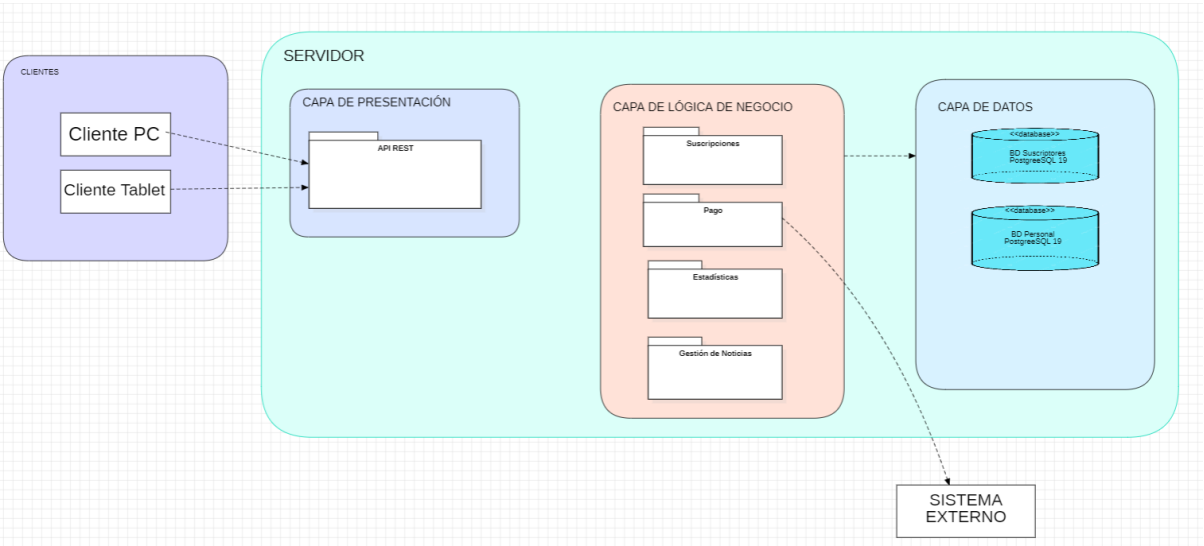
Por último, este estilo permite modificar una capa sin afectar a las demás y aplicar patrones de diseño dentro de cada una en las siguientes iteraciones.

Posteriormente se planteó un debate sobre si el estilo Por Capas era suficiente por sí solo o si debía emplearse también el estilo REST. El anexo establece que las peticiones se realizan mediante protocolos HTTP/REST (RD4), por lo que el estilo REST respondería directamente a los requisitos en los que el cliente interactúa con el servidor, como el alta y la baja anónima de suscripciones (RF2, RF3). Además, al llegar cada petición por HTTP, el servidor puede identificar el tipo de dispositivo del que procede (RF1).

Sin embargo, se observó que REST solo define la comunicación entre el cliente y el servidor y no cómo organizar internamente los módulos de suscripciones, pagos y estadísticas, por lo que no era suficiente por sí solo para estructurar la arquitectura. A su vez, el estilo Por Capas no define cómo se comunica el cliente con el servidor, que era una exigencia de nuestro problema.

Debido a estos motivos, los Arquitectos Software Cognitivos junto con los Arquitectos Software Senior se pusieron de acuerdo en emplear el estilo Por Capas junto con el estilo REST, de forma que las capas organizan la estructura interna del sistema y REST se encarga de la comunicación entre el cliente y el servidor mediante HTTP.

Diagrama de Paquetes (Estilo por Capas + REST)



Repositorio GitHub

https://github.com/rimafd/DAS-Pr-ctica1